

Análise léxica e sintática com PLY (Python Lex-Yacc)

Estudo da ferramenta, mini-manual de uso e o analisador léxico da linguagem Elgol

Trabalho final da disciplina de Compiladores

Integrantes do grupo: _____

Professor: Elgio Schlemer

Setembro de 2026

Resumo

O `PLY` é uma versão em Python do `lex` e do `yacc`, as ferramentas clássicas para gerar analisador léxico e sintático. O artigo compara o `PLY` com as principais alternativas, mostra como ele funciona por dentro e aponta o que ele tem de bom e de ruim. Um dos pontos ruins pesa bastante: o autor arquivou o projeto em dezembro de 2025. Para testar na prática, o grupo escreveu um analisador léxico para C e um sintático `LALR(1)` para um pedaço da linguagem. Na Etapa 1 do projeto da disciplina veio a parte mais trabalhosa, um analisador léxico para Elgol, linguagem criada pelo professor, cujas regras quebram o jeito mais comum de usar a ferramenta. Tudo o que aparece nas figuras saiu de execução real do código.

Palavras-chave: compiladores; análise léxica; análise sintática; `LALR(1)`; `PLY`; Python.

1. Introdução

Todo mundo que já precisou tratar log de servidor escreveu um parser sem perceber. Separar os campos de uma linha de log do Nginx, pegar o IP de origem na saída de um comando de roteador, ler um arquivo de configuração para descobrir qual porta está liberada. Quase sempre sai um split, duas ou três expressões regulares e um for. Resolve.

Até o formato crescer. Quando aparece aninhamento, bloco dentro de bloco, exceção para um caso específico, aquele script de trinta linhas vira uma coisa que ninguém tem coragem de mexer.

Compilador lida com esse problema há décadas, e de um jeito organizado. O trabalho é dividido em etapas. A análise léxica lê a sequência de caracteres e devolve tokens, que são as unidades mínimas da linguagem: um nome de variável, um número, um operador, uma palavra reservada. A análise sintática recebe esses tokens, confere se a ordem deles respeita a gramática e monta uma árvore que representa o programa.

As duas etapas têm descrição formal bem estabelecida. Expressão regular para a primeira, gramática livre de contexto para a segunda. Por isso foram as primeiras partes do compilador a serem automatizadas, ainda nos anos 70, com o *lex* e o *yacc* dos laboratórios Bell. De lá para cá quase toda linguagem popular ganhou sua versão dessas ferramentas.

O grupo ficou com o *PLY*. O motivo é prático: Python é a linguagem que o grupo inteiro domina melhor, e o *PLY* é a implementação mais conhecida para ela.

2. As ferramentas pesquisadas

Quase tudo na Tabela 1 descende direto do *lex* e do *yacc*. O *lex* transforma expressões regulares em um autômato finito. O *yacc* transforma uma gramática livre de contexto em uma tabela LALR(1). A maioria das ferramentas de hoje usa esses mesmos algoritmos, então não é aí que elas se diferenciam.

O que muda na prática é **o que a ferramenta entrega no final**.

Flex, JFlex e Goyacc geram código. Você escreve a especificação num arquivo separado, roda o gerador, ele produz um arquivo em C ou Java e esse arquivo entra na compilação junto com o resto do projeto. Tem um passo de build no meio. O *PLY* não trabalha assim. O analisador é montado na hora em que o programa roda, dentro do próprio processo Python, sem arquivo intermediário nenhum.

Ferramenta	Linguagem gerada ou hospedeira	Papel	Algoritmo	Observações
Lex e Flex	C (ANSI)	Léxico	AFD a partir de ER	Padrão histórico do Unix. O Flex é a versão livre, ainda mantida e muito usada. Gera <i>lex.yy.c</i> e costuma ser combinado com Yacc ou Bison.
Flex++	C++	Léxico	AFD a partir de ER	Variante que gera uma classe C++ no lugar de funções globais. Ajuda em reentrância e quando o mesmo programa precisa de vários analisadores.
CSLex	C#	Léxico	AFD a partir de ER	Porte do JLex para .NET, que por sua vez é porte do Lex. Projeto antigo e com pouca atividade.
JFlex	Java	Léxico	AFD a partir de ER	Sucessor do JLex, maduro e ativo. Suporta Unicode completo e normalmente é usado junto com CUP ou BYacc/J na parte sintática.
Jison	JavaScript	Léxico e sintático	LALR(1), SLR, LR(1), LL	Porte do Bison para JavaScript. Aceita arquivo de gramática no formato do Bison e roda tanto em Node.js quanto direto no browser.

Ferramenta	Linguagem gerada ou hospedeira	Papel	Algoritmo	Observações
PLY	Python	Léxico e sintático	LALR(1)	Reimplementação de lex e yacc em Python puro, com a especificação escrita dentro do próprio código. Arquivado pelo autor em dezembro de 2025.
Goyacc	Go	Sintático	LALR(1)	Porte do yacc distribuído com as ferramentas Go. Cobre só a parte sintática. O léxico costuma ser escrito à mão ou com o pacote text/scanner.

Tabela 1. Comparativo das ferramentas de análise léxica e sintática pesquisadas.

Tem uma confusão comum aqui, e o enunciado do TR4 toca nela. A linguagem em que a ferramenta foi escrita define em que linguagem você programa as ações semânticas. Ela não limita o que pode ser analisado. O PLY é Python, então o compilador sai em Python, mas ele consegue ler C, SQL, JSON ou qualquer formato que alguém especificar. Na Seção 6 ele lê C. Na Seção 7, uma linguagem que não existia antes deste semestre.

3. A ferramenta escolhida

3.1 De onde veio e como está hoje

David Beazley escreveu o PLY em 2001 para as aulas de compiladores que dava na Universidade de Chicago. A ideia era didática. Ele queria que os alunos usassem lex e yacc de verdade, sem uma camada de abstração escondendo a teoria. A versão estável é a **3.11**, de 2018, a última publicada no PyPI e a que vem com `pip install ply`. Existe uma 4.0 no repositório do autor, que pede Python 3.6 ou mais novo.

Tem um porém sério. Em 21 de dezembro de 2025 o autor arquivou o repositório oficial. Ficou tudo em modo somente leitura, e no README ele avisa que abandonou o projeto depois de 25 anos, recomendando outra biblioteca para quem for começar algo novo. Não sai mais correção de bug, nem de segurança.

Isso é mais comum do que parece. Tem muito software sem manutenção rodando em produção por aí, e a decisão de manter ou trocar depende de onde ele está e do risco que traz. Para a disciplina, o impacto é zero: o código é estável, não depende de nada externo e roda normalmente no Python atual (os testes deste artigo foram feitos no 3.12). Num sistema que vai ficar anos no ar, é um risco que precisa ser assumido sabendo o que se está fazendo. A Seção 8 volta nesse assunto.

3.2 Como o PLY funciona por dentro

São só dois módulos. O `ply.lex` faz o papel do lex e o `ply.yacc`, o do yacc. A Figura 1 mostra o caminho da entrada até a árvore.

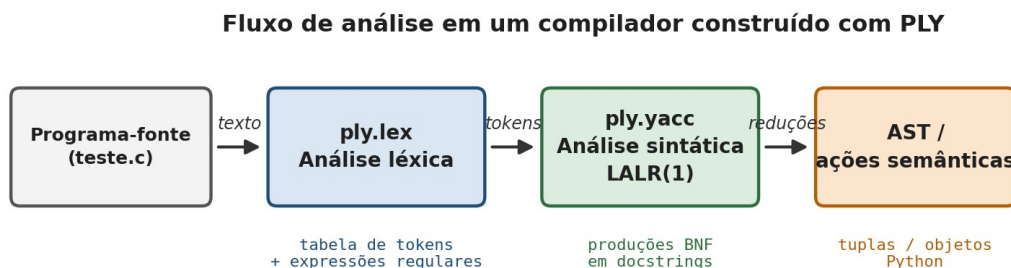


Figura 1. Fluxo de análise em um compilador construído com PLY.

A grande diferença em relação ao resto da tabela é que **não existe arquivo de especificação**. Nada de `.l` ou `.y`. As expressões regulares e as produções da gramática ficam dentro do próprio código Python: as regras léxicas em variáveis e funções com prefixo `t_`, as sintáticas em docstrings de funções com prefixo `p_`. Quando o programa chama `lex.lex()` ou `yacc.yacc()`, o PLY procura esses nomes no módulo e monta os analisadores em memória. A Figura 2 resume esse processo.

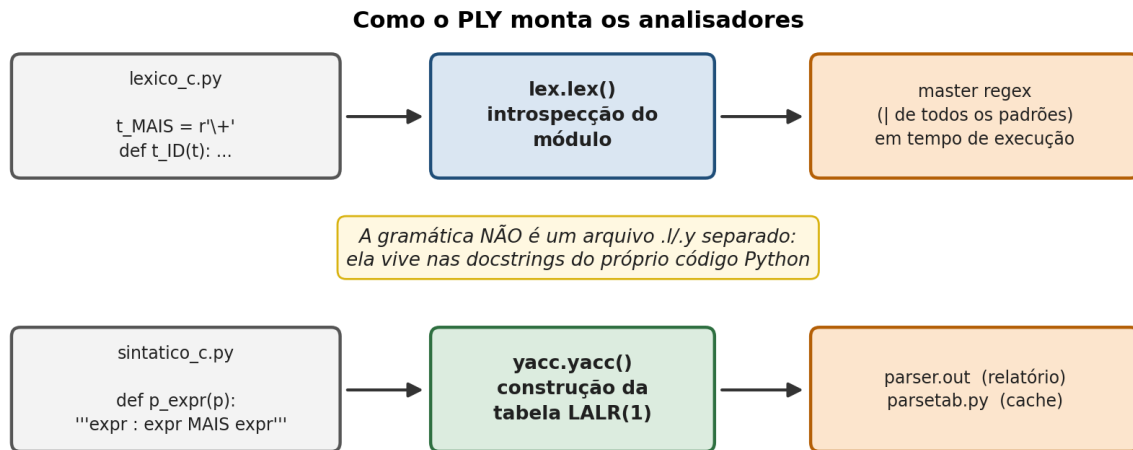


Figura 2. Construção dos analisadores por introspecção do módulo Python.

No lado léxico, todos os padrões viram uma expressão regular gigante, com as alternativas separadas por barra vertical, e quem percorre a entrada é o módulo `re` da biblioteca padrão. No lado sintático, o PLY calcula a tabela LALR(1), grava um relatório legível no `parser.out` e salva a tabela pronta no `parsetab.py`. Enquanto a gramática não muda, as próximas execuções reaproveitam esse arquivo. Calcular a tabela é lento. Usar a tabela pronta é rápido.

4. Mini-manual: o módulo ply.lex

4.1 Instalação

```
# via pip (versao 3.11, a que esta publicada no PyPI)
pip install ply

# alternativa recomendada pelo autor: copiar o diretorio ply/
# (lex.py e yacc.py) para dentro do proprio projeto.
```

Como não tem dependência externa, dá para simplesmente copiar os dois arquivos para dentro do projeto. Isso tem nome, *vendoring*, e passou a fazer sentido agora que o pacote não recebe mais manutenção: se o PLY sumir do PyPI um dia, o projeto continua de pé. Para a disciplina, o pip basta.

4.2 Estrutura mínima de um analisador léxico

Três coisas são obrigatórias. Uma lista chamada `tokens`, com o nome de cada tipo de token. Uma regra para cada um deles. E a chamada `lex.lex()`. O exemplo abaixo roda do jeito que está.

```
import ply.lex as lex

tokens = ['NUMERO', 'MAIS', 'VEZES']

t_MAIS = r'\+'          # regra simples: so a expressao regular
t_VEZES = r'\*'

def t_NUMERO(t):         # regra com funcao: permite acao semantica
    r'\d+'
    t.value = int(t.value)
    return t

t_ignore = ' \t'

def t_error(t):
    print("Caractere ilegal: %s" % t.value[0])
    t.lexer.skip(1)

lexer = lex.lex()
lexer.input("3 + 4 * 10")
for tok in lexer:
    print(tok.type, tok.value, tok.lineno, tok.lexpos)
```

Cada token é um objeto `LexToken` com quatro atributos: `type` (o nome do token), `value` (o texto reconhecido, ou o valor já convertido pela ação semântica), `lineno` (a linha) e `lexpos` (a posição em caracteres desde o começo da entrada, contando do zero).

4.3 A ordem em que as regras são testadas

É aqui que mais gente tropeça na primeira vez. Com a ordem errada, o analisador reconhece `"if"` como nome de variável e ninguém percebe até alguma coisa quebrar lá na frente.

Ordem	Tipo de regra	Critério de ordenação
1º	Regra definida como função (def t_NOME)	Na ordem em que aparece no arquivo. Quem escreve controla a prioridade colocando a função mais específica antes.
2º	Regra definida como string (t_NOME = r'...')	Por comprimento decrescente da expressão regular. É por isso que "<=" é testado antes de "<" sem nenhuma configuração extra.

Tabela 2. Precedência entre as regras léxicas no PLY.

Duas consequências práticas. Se um token é prefixo de outro, a posição dele no arquivo importa. No analisador de C deste trabalho, FLOAT_CONST vem antes de INT_CONST, porque na ordem contrária o 3.5 seria lido como o inteiro 3, um ponto e o inteiro 5. E como no fim tudo vira uma expressão regular só, trocar a ordem das regras muda o resultado da análise. Não só a velocidade.

4.4 Palavras reservadas

O primeiro impulso é criar uma regra para cada palavra reservada: `t_IF = r'if'`, `t_ELSE = r'else'` e assim por diante. Não funciona bem. A palavra "ifelse" viraria IF seguido de ELSE, e a expressão regular principal ia crescer à toa, deixando tudo mais lento. O jeito que a documentação recomenda é reconhecer qualquer palavra como identificador e trocar o tipo depois, olhando num dicionário:

```
reservadas = {'if': 'IF', 'else': 'ELSE', 'while': 'WHILE', ...}

tokens = ['ID', 'INT_CONST', ...] + list(reservadas.values())

def t_ID(t):
    r'[A-Za-z_][A-Za-z0-9_]*'
    t.type = reservadas.get(t.value, 'ID')    # troca o tipo do token
    return t
```

Assim o autômato sempre pega a palavra mais longa possível, que é o comportamento certo para C, e descobrir se ela é reservada custa uma consulta de dicionário. (Na Seção 7 aparece uma linguagem em que essa receita não basta.)

4.5 Descartar espaço, comentário e diretiva

Há dois mecanismos. A variável `t_ignore` recebe uma string com os caracteres que devem ser pulados sem chamar função nenhuma, e é por ela que passam espaço e tabulação. Para o resto, basta uma regra que **não retorna token**. Sem return, o texto reconhecido é jogado fora.

```
t_ignore = ' \t\r'

def t_COMENTARIO_BLOCO(t):
    r'/*(.*\n)*?*/'          # o ? deixa o quantificador preguiçoso
    t.lexer.lineno += t.value.count('\n')    # nao perder a contagem
    # sem "return t": o comentario e descartado

def t_COMENTARIO_LINHA(t):
    r'//[^\n]*'
    pass
```

O quantificador preguiçoso (`*?`) faz diferença real. Na versão gulosa, se o arquivo tiver dois comentários, tudo entre o primeiro `/*` e o último `*/` vira um comentário só, e o código no meio some sem aviso nenhum. A outra pegadinha é o `lineno`. O PLY não conta linhas sozinho, então um comentário de dez linhas descartado sem atualizar o contador faz todas as mensagens de erro seguintes apontarem para a linha errada.

4.6 Linha e coluna na mensagem de erro

O `lexpos` vem pronto. A coluna, não. A conta é rápida e compensa fazer desde o começo, porque mensagem de erro sem coluna é bem mais chata de usar:

```
def t_nova_linha(t):
    r'\n+'
    t.lexer.lineno += len(t.value)    # sem return: apenas conta

def coluna(entrada, token):
    inicio = entrada.rfind('\n', 0, token.lexpos) + 1
    return (token.lexpos - inicio) + 1
```

4.7 Erro léxico

Quando nenhum padrão casa, o PLY chama `t_error`. O normal é mostrar o problema e chamar `t.lexer.skip(1)`, que pula um caractere e segue. Assim todos os erros do arquivo aparecem numa passada só, em vez de o analisador parar no primeiro.

4.8 Estados léxicos

Algumas construções pedem um modo de leitura diferente, tipo comentário aninhado ou string com escape complicado. Para isso existem os estados. Eles são declarados na variável `states` e ativados com `lexer.begin()` ou `lexer.push_state()`. Enquanto um estado está ativo, só valem as regras com o prefixo dele:

```
states = (('comentario', 'exclusive'),)

def t_inicia_comentario(t):
    r'\/\*'
    t.lexer.push_state('comentario')    # entra no estado

def t_comentario_fim(t):
    r'\*\/'
    t.lexer.pop_state()                 # volta ao estado anterior

t_comentario_ignore = ' \t'
def t_comentario_error(t):
    t.lexer.skip(1)
```

Um estado `exclusive` ignora as regras do estado padrão. Um `inclusive` soma as duas. Comentário comum de C não precisa disso, já que uma regex preguiçosa resolve. Estado vira necessidade quando a construção não é regular, e comentário aninhado é o exemplo típico, porque aninhamento não se descreve com expressão regular.

5. Mini-manual: o módulo ply.yacc

5.1 Produção dentro da docstring

Cada função `p_` tem na docstring uma ou mais produções em BNF. O parâmetro `p` funciona como uma lista. O `p[0]` é o valor do lado esquerdo, que a função precisa preencher. O `p[1]`, o `p[2]` e os seguintes são os valores do lado direito, na ordem em que aparecem.

```
def p_expressao_soma(p):
    '''expressao : expressao MAIS termo'''
    #           p[1]      p[2] p[3]
    p[0] = ('bin', '+', p[1], p[3])

def p_expressao_termo(p):
    '''expressao : termo'''
    p[0] = p[1]
```

Alternativas do mesmo não terminal podem ir na mesma docstring, separadas por `"|"`. Produção vazia se escreve com um não terminal `empty` sem corpo. Se nada for dito, o símbolo inicial da gramática é o lado esquerdo da primeira produção que o PLY encontrar.

5.2 Precedência e associatividade

Dá para escrever a gramática de expressões já dividida em níveis (expressão, termo e fator). É o jeito de livro. Funciona, mas fica comprido, e a árvore sai cheia de nós intermediários que não carregam informação nenhuma. O caminho usado aqui foi outro: uma regra binária única, ambígua de propósito, com a ambiguidade resolvida pela tabela precedence. A ordem vai **do menor para o maior** nível:

```
precedence = (
    ('right', 'ATRIB'),
    ('left', 'OU_LOGICO'),
    ('left', 'E_LOGICO'),
    ('left', 'IGUAL', 'DIFERENTE'),
    ('left', 'MENOR', 'MAIOR', 'MENORIGUAL', 'MAIORIGUAL'),
    ('left', 'MAIS', 'MENOS'),
    ('left', 'VEZES', 'DIVIDE', 'MODULO'),
    ('right', 'NAO_LOGICO', 'UMENOS'),
)
```

O `UMENOS` não é um token real. Ele só existe para dar ao menos unário uma precedência diferente da do menos binário, e entra na produção pela diretiva `%prec`:

```
def p_expressao_unaria(p):
    '''expressao : MENOS expressao %prec UMENOS'''
    p[0] = ('un', '-', p[2])
```

Sem ele, $-2 * 3$ seria lido como $-(2 * 3)$. A associatividade também muda o resultado. Com a subtração declarada como `left`, $10 - 3 - 2$ dá 5. Se fosse `right`, daria 9.

5.3 Conflito e o relatório parser.out

Quando a gramática é ambígua e a precedência não cobre o caso, o PLY avisa que houve conflito *shift/reduce* ou *reduce/reduce* e detalha tudo no `parser.out`. A gramática deste trabalho tem um conflito só, o velho conhecido *dangling else*. No trecho `"if (a) if (b) X; else Y;"`, o `else` pode ser de qualquer um dos dois `if`. A Figura 3 traz o trecho do relatório.

```

trecho de parser.out: conflito shift/reduce (dangling else)
state 107
  (21) comando -> IF ABRE_PAR expressao FECHA_PAR comando .
  (22) comando -> IF ABRE_PAR expressao FECHA_PAR comando . ELSE com
! shift/reduce conflict for ELSE resolved as shift
FECHA_CHAVE      reduce using rule 21 (comando -> IF ABRE_PAR expre
IF                reduce using rule 21 (comando -> IF ABRE_PAR expre
WHILE            reduce using rule 21 (comando -> IF ABRE_PAR expre
...

```

Figura 3. Trecho do parser.out com o conflito shift/reduce do dangling else.

O relatório diz que o conflito foi resolvido com **shift**, ou seja, o `else` fica com o `if` mais próximo. É o que o padrão da linguagem C define. Só que tem um detalhe que pega muita gente: aviso de conflito não impede a geração do parser. *Shift/reduce* sempre vira *shift*, *reduce/reduce* sempre fica com a regra declarada primeiro, tudo em silêncio. O aviso quer dizer que a ferramenta decidiu por você. Se a decisão foi a certa, quem precisa conferir é você.

5.4 Erro sintático e recuperação

A função `p_error(p)` é chamada quando o token lido não se encaixa em nenhum estado. Mostrar o erro e seguir em frente parece fácil, e o jeito mais óbvio dá resultado ruim. A primeira versão deste trabalho chamava `parser.errok()` dentro de `p_error`. Resultado: um arquivo com dois erros reais gerou **dez** mensagens, porque cada token depois do erro era rejeitado isoladamente e o parser nunca voltava a se alinhar com a entrada.

O certo é usar o token especial `error` dentro de uma produção. Quando o erro acontece, o parser vai tirando símbolos da pilha até conseguir casar essa produção. Na teoria, é a *recuperação em modo pânico*. Aqui o ponto e vírgula faz o papel de ponto de sincronização:

```

def p_comando_erro(p):
    '''comando : error PONTO_VIRGULA'''
    p[0] = ('comando_invalido',)

def p_error(p):
    if p:
        print("[ERRO SINTATICO] token inesperado '%s' na linha %d"
              % (p.value, p.lineno))
        # NAO chamar parser.errok() aqui: isso cancela o modo de
        # recuperacao e produz erro em cascata.
    else:
        print("[ERRO SINTATICO] fim de arquivo inesperado")

```

Com essa produção, o mesmo arquivo passou a mostrar três mensagens: uma do erro léxico e duas dos erros sintáticos que existiam de fato (Figura 4).

```

$ python sintatico_c.py erro.c (apenas as linhas de erro)
[ERRO LEXICO] caractere ilegal '@' na linha 2, coluna 16
[ERRO SINTATICO] token inesperado '3' (INT_CONST) na linha 2
[ERRO SINTATICO] token inesperado ')' (FECHA_PAR) na linha 3

```

Figura 4. Detecção e recuperação de erros em um arquivo propositalmente inválido.

5.5 Opções úteis de `yacc.yacc()`

Parâmetro	Efeito
<code>debug=True</code> (padrão)	Gera o <code>parser.out</code> com os estados do autômato LALR, as produções numeradas e os conflitos. Em produção, desligar.
<code>write_tables=False</code>	Não grava o <code>parsetab.py</code> . Útil enquanto a gramática está mudando muito, para não correr o risco de rodar com tabela velha em cache.
<code>start='símbolo'</code>	Troca o símbolo inicial da gramática. Prático para testar um não terminal isolado.
<code>errorlog=yacc.NullLogger()</code>	Silencia os avisos, inclusive os de token declarado e não usado. Usar com cuidado, porque esses avisos costumam apontar erro real.
<code>picklefile=...</code>	Grava a tabela em formato pickle. Ver o alerta de segurança na Seção 8.

Tabela 3. Principais parâmetros de configuração de `yacc.yacc()`.

6. Estudo de caso: um analisador para C

O primeiro teste foi com C, que era o que o TR4 pedia. O léxico cobre as 32 palavras reservadas do C89, inteiros em decimal, octal e hexadecimal, ponto flutuante com notação científica, caractere, string, todos os operadores, os dois tipos de comentário e as diretivas de pré-processador. O sintático reconhece um pedaço da linguagem, que inclui definição de função, declaração, atribuição, expressão, if/else, while, return e chamada de função. A saída dele é a AST.

6.1 As regras léxicas para C

A Tabela 4 lista cada classe de token com a expressão regular correspondente. As regras que são funções aparecem na mesma ordem do arquivo, que é a ordem em que o PLY testa.

Classe	Token	Expressão regular ou mecanismo
Palavras reservadas	INT, FLOAT, IF, WHILE... (32)	Dicionário consultado dentro de t_ID (Seção 4.4)
Identificador	ID	[A-Za-z_][A-Za-z0-9_]*
Ponto flutuante	FLOAT_CONST	((\d+\.\d*) (\.\d+))([eE][-+]?[fF\LL]? \d+[eE][-+]?[fF\LL]?)
Inteiro (dec, oct, hex)	INT_CONST	0[xX][0-9a-fA-F]+[uU\LL]* 0[0-7]+[uU\LL]* \d+[uU\LL]*
Caractere	CHAR_CONST	'([\^\\\'\\n] \\\.)*'
String	STRING_LITERAL	"([\^\\\'\\n] \\\.)*"
Aritméticos	MAIS, MENOS, VEZES, DIVIDE, MODULO	\+ - * / %
Incremento e decremento	INCREMENTO, DECREMENTO	\+\+ --
Atribuição composta	MAIS_ATRIB, MENOS_ATRIB, VEZES_ATRIB, DIVIDE_ATRIB	\+= -= *= /=
Relacionais	IGUAL, DIFERENTE, MENOR, MAIOR, MENORIGUAL, MAIORIGUAL	== != < > <= >=
Lógicos	E_LOGICO, OU_LOGICO, NAO_LOGICO	&& \ \ !
Bit a bit	E_BIT, OU_BIT, XOR, NOT_BIT, SHIFT_ESQ, SHIFT_DIR	& \ \^ ~ << >>
Delimitadores	PONTO_VIRGULA, VIRGULA, PONTO, SETA, parênteses, chaves, colchetes	; , \. -> () { } []
Descartados	(nenhum)	Espaço (t_ignore), // linha, /* bloco */ (preguiçoso), # diretiva

Tabela 4. Regras léxicas implementadas para a linguagem C.

Sobre os números, a ordem importa. t_FLOAT_CONST está declarada antes de t_INT_CONST, e como as duas são funções, o PLY testa nessa ordem. Invertendo, "3.5e2" viraria INT_CONST(3), PONTO, INT_CONST(5) e ID("e2"). Nenhum aviso. O erro só aparece para quem olhar a saída com atenção.

6.2 Entrada e saída do analisador léxico

```
/* teste.c */
#include <stdio.h>

int fatorial(int n) {
    int resultado = 1;
    while (n > 1) {
        resultado *= n;
    }
}
```

```

        n = n - 1;
    }
    return resultado;
}

int main(void) {
    float media = 3.5e2;      // constante em notacao cientifica
    char letra = 'A';
    int mascara = 0xFF;
    if (media >= 100.0 && mascara != 0) {
        printf("ok %d\n", fatorial(5));
    } else {
        media = media / 2;
    }
    return 0;
}

```

```
$ python lexico_c.py teste.c
```

LINHA	TOKEN	LEXEMA/VALOR	COL
4	INT	'int'	1
4	ID	'fatorial'	5
4	ABRE_PAR	'('	13
4	INT	'int'	14
4	ID	'n'	18
4	FECHA_PAR	')'	19
4	ABRE_CHAVE	'{'	21
5	INT	'int'	5
5	ID	'resultado'	9
5	ATRIB	'='	19
...	(saída truncada)		

Total de tokens reconhecidos: 90

Figura 5. Saída do analisador léxico de C, com 90 tokens reconhecidos.

As ações semânticas já entregam os valores convertidos: 0xFF sai como 255 e 3.5e2 como 350.0. O #include e os comentários foram descartados e, mesmo assim, a contagem de linhas bateu. O primeiro token, int, aparece na linha 4, que é onde ele está no arquivo.

6.3 Gramática e árvore sintática

A gramática tem 31 produções. O trecho abaixo é o de comandos, de onde vem o conflito da Seção 5.3:

```

def p_comando_if(p):
    '''comando : IF ABRE_PAR expressao FECHA_PAR comando
               | IF ABRE_PAR expressao FECHA_PAR comando ELSE comando'''
    p[0] = ('if', p[3], p[5], p[7] if len(p) == 8 else None)

def p_comando_while(p):
    '''comando : WHILE ABRE_PAR expressao FECHA_PAR comando'''
    p[0] = ('while', p[3], p[5])

def p_expressao_binaria(p):
    '''expressao : expressao MAIS expressao
                 | expressao MENOS expressao
                 | expressao VEZES expressao

```

```
        | ... (13 operadores na mesma producao)'''  
p[0] = ('bin', p[2], p[1], p[3])
```

Cada redução cria um nó da AST na forma de uma tupla do Python. A Figura 6 é a árvore do arquivo de teste.

```
$ python sintatico_c.py teste.c  
AST gerada:  
  
programa  
  funcao  
    'int'  
    'fatorial'  
    int  
      'n'  
    bloco  
      declara  
        'int'  
        'resultado'  
        const  
          1  
      while  
        bin  
          '>'  
          id  
            'n'  
          const  
            1  
        bloco  
          expr  
            atrib  
              '*='  
              'resultado'  
            id  
              'n'  
          expr  
            atrib  
              ... (subárvore de main omitida) ...
```

Figura 6. AST produzida pelo analisador sintático (subárvore de main omitida).

Dá para ver que a precedência funcionou. Em $n = n - 1$, a atribuição ficou na raiz e a subtração embaixo dela. Na condição $media \geq 100.0 \ \&\& \ mascara \neq 0$, o $\&\&$ ficou acima das duas comparações.

7. Etapa 1: o analisador léxico de Elgol

Elgol é a linguagem que o professor definiu para o projeto final. Parece simples. Pelo menos até agora ela não tem string nem ponto flutuante, e não tem nem o número zero. Só que duas regras dela quebram o jeito mais direto de escrever o analisador, e o léxico precisou de uma estrutura bem diferente da usada para C.

7.1 As regras da linguagem

Classe	Token	Regra
Identificador	ID	Consoante maiúscula, depois duas ou mais letras, terminando em letra minúscula. Mínimo de 4 caracteres e só letras.
Nome de função	FUNCAO	Cifrão seguido de um identificador válido. Token próprio, separado de ID.
Número inteiro	NUMERO	Primeiro dígito diferente de zero, seguido de dígitos. Não existe o número zero, que é representado pelo operador <code>_Z_</code> .
Palavras reservadas	ELGIO, DECIMAL, ZERO, NEG, EXP, RESTO, ENQUANTO, SE, ENTAO, SENAO, PARA, INICIO, FIM, MAIOR, MENOR, IGUAL, DIFERENTE, MIGUAL, MAIGUAL	Lista fechada e sensível a caixa. Os tokens ZERO e NEG correspondem a <code>_Z_</code> e <code>_NEG_</code> . MIGUAL é "migual" e MAIGUAL é "Migual".
Operadores	ATRIB, MAIS, MENOS, DIVIDE, VEZES, EXP, RESTO	Atribuição (=), soma (+), subtração (-), divisão (/) e multiplicação, que é a letra x. EXP e RESTO são operadores escritos como palavra.
Delimitadores	PONTO, VIRGULA, ABRE_PAR, FECHA_PAR	O ponto encerra cada comando, no lugar do ponto e vírgula. Parênteses para parâmetro de função.
Comentário	(descartado)	O asterisco abre comentário que vai até o fim da linha.
Ignorados	(descartado)	Espaço, tabulação e quebra de linha.

Tabela 5. Classes de token da linguagem Elgol.

7.2 Onde o dicionário não resolve sozinho

A receita da Seção 4.4 parte de uma premissa: toda palavra reservada tem cara de identificador. Em C é verdade. `while` casa com o padrão de identificador, então basta reconhecer como identificador e corrigir o tipo depois.

Em Elgol, quase nenhuma reservada casa com a regra de identificador. Algumas começam com minúscula (`inicio`, `se`, `maior`), outras são todas maiúsculas (`DECIMAL`, `EXP`) e duas começam com underscore (`_Z_` e `_NEG_`). A regra exige consoante maiúscula no começo e minúscula no fim, e nenhuma dessas passa.

Nenhuma, menos uma. `MIgual` começa com M, tem seis letras e termina em "l" minúsculo. Pela regra, é um identificador perfeito. Se o analisador testar a regra de identificador antes de olhar a lista de reservadas, `MIgual` vira nome de variável e um operador de comparação some da linguagem sem ninguém perceber.

7.3 Por que a validação fica fora da expressão regular

A regra de identificador cabe inteira numa regex: consoante maiúscula, duas ou mais letras, minúscula no final. Escrever só isso e seguir em frente é tentador. O problema está no que acontece com a entrada que não casa.

Um exemplo é `Teste39`. Com essa regex, o PLY reconhece "Teste" como identificador válido e depois "39" como número. Dois tokens corretos, zero erro. Só que o enunciado diz com todas as letras que

Teste39 não é identificador. Com número acontece a mesma coisa: 034 vira um erro no zero seguido de um 34 válido, quando deveria ser um erro só. E Tes_Te se desmancha em pedaços ainda piores.

A solução foi inverter a lógica. O padrão da regra é genérico e engole qualquer coisa com cara de palavra, com dígito e underscore incluídos. Quem decide se a palavra é válida é o código Python dentro da função, e a primeira coisa que ele faz é olhar a lista de reservadas:

```
def t_ID(t):
    r'[A-Za-z_][A-Za-z0-9_]*'      # padrao generico de palavra
    if t.value in reservadas:      # reservada tem prioridade
        t.type = reservadas[t.value]
        return t
    motivos = critica_identificador(t.value)
    if motivos:
        reporta(t, motivos)
        return None                # descarta o token e segue
    t.type = 'ID'
    return t
```

Número e nome de função seguem a mesma estrutura, com os padrões `[0-9][A-Za-z0-9_]*` e `\$[A-Za-z0-9_]*`. Sai mais código do que uma regex sozinha. Em troca, cada lexema problemático vira um erro só, com a explicação do que está errado nele. Falta um ponto fácil de esquecer: a multiplicação em Elgol é a letra x. Ela precisa estar no dicionário das reservadas, senão o analisador acusa um identificador inválido de uma letra só.

7.4 O que o enunciado deixou em aberto

Alguns pontos da especificação ficaram em aberto. Nesses casos o grupo tomou uma decisão e registrou o motivo na Tabela 6. Todas as decisões ficam em constantes no começo do arquivo, então mudar qualquer uma delas é alterar uma linha.

Ponto em aberto	Decisão	Motivo
A letra Y conta como consoante?	Sim. As vogais são A, E, I, O e U.	Nenhum exemplo do enunciado depende disso. O caso "Xyz" foi recusado por tamanho, não pela letra Y.
Letra acentuada é letra?	Não. Cai como caractere ilegal.	O alfabeto adotado é ASCII. Nenhum exemplo do enunciado usa acento dentro de identificador.
A vírgula é um símbolo da linguagem?	Sim, com token próprio.	Não aparece na lista de símbolos do enunciado, mas aparece duas vezes no exemplo de código do professor, na declaração e na chamada da função.
Parar no primeiro erro léxico?	Não. Reporta todos e segue.	O exemplo do professor tem um erro proposital no meio do arquivo e o código continua depois dele.
Número de um dígito só é válido?	Sim.	O exemplo usa "3 EXP 4" e "13 RESTO 4".
mígal e Mígal são a mesma palavra?	Não. São duas palavras reservadas distintas.	A lista do enunciado traz as duas grafias separadas, o que só faz sentido se a linguagem diferenciar maiúscula de minúscula.

Tabela 6. Pontos em aberto na especificação e as decisões adotadas.

7.5 Resultado no exemplo do professor

O exemplo do enunciado gera 75 tokens e um único erro léxico, no Vim. É a mesma linha que o comentário do professor aponta. A Figura 7 mostra um recorte da saída.

```

$ python lexico_elgol.py exemplo_professor.elg
LINHA  COL      TOKEN      LEXEMA
-----
5      1         DECIMAL    'DECIMAL '
5      9         FUNCAO     '$Soma '
5      15        ABRE_PAR   '('
5      16        DECIMAL    'DECIMAL '
5      24        ID         'Numm'
5      28        VIRGULA   ','
5      30        DECIMAL    'DECIMAL '
5      38        ID         'Dois'
5      42        FECHA_PAR ')'
5      44        PONTO     '.'
...      (saida truncada)
14     3         DECIMAL    'DECIMAL '
[ERRO LEXICO] linha 14, coluna 11: 'Vim'
              nao e um identificador valido:
              precisa de no minimo 4 caracteres (tem 3)
14     15        PONTO     '.'
16     3         ID         'Lixo'
16     8         ATRIB     '='
...      (saida truncada)
Tokens reconhecidos: 75
Erros lexicos: 1

```

Figura 7. Saída do analisador no exemplo de código fornecido pelo professor.

Tem uma coisa curiosa nesse exemplo. O parâmetro da função se chama `Dois`, mas o corpo usa `Doiss`. Os dois nomes são identificadores válidos, então o analisador léxico não reclama, e está certo em não reclamar. Variável não declarada é assunto da análise semântica, que fica para outra etapa. O mesmo vale para `Resultado`, usado sem declaração nenhuma.

7.6 Bateria de testes

Um exemplo só não prova muita coisa. O grupo montou um segundo arquivo com todos os casos que o enunciado cita, válidos e inválidos, e colocou 16 erros de propósito. O analisador encontra os 16. Na Figura 8, `Teste39` e `034` aparecem cada um como um único erro. Quebrar esses casos em dois tokens era justamente o problema descrito na Seção 7.3.

```

$ python lexico_elgol.py casos_borda.elg (recorte: erros e resumo final)
[ERRO LEXICO] linha 11, coluna 3: 'Teste39'
    nao e um identificador valido:
    so pode conter letras (encontrado: '3' '9')
    deve terminar com letra minuscula
[ERRO LEXICO] linha 13, coluna 3: 'Xyz'
    nao e um identificador valido:
    precisa de no minimo 4 caracteres (tem 3)
[ERRO LEXICO] linha 16, coluna 3: 'Letra'
    nao e um identificador valido:
    deve terminar com letra minuscula
[ERRO LEXICO] linha 17, coluna 3: 'Ateras'
    nao e um identificador valido:
    deve comecar com consoante, e 'A' e vogal
...
[ERRO LEXICO] linha 21, coluna 11: '034'
    numero nao pode comecar com 0 (use o operador _Z_ para representar zero)
[ERRO LEXICO] linha 23, coluna 11: '12abc'
    numero so pode conter digitos (encontrado: 'a' 'b' 'c')
[ERRO LEXICO] linha 29, coluna 11: '$Te'
    nome de funcao invalido:
    precisa de no minimo 4 caracteres (tem 2)
...
Tokens reconhecidos: 111
Erros lexicos: 16

```

Figura 8. Diagnóstico dos casos inválidos listados no enunciado.

7.7 Como executar

```

pip install ply

python lexico_elgol.py exemplo_professor.elg      # lista os tokens
python lexico_elgol.py casos_borda.elg            # bateria de testes
python lexico_elgol.py arquivo.elg --resumo        # so a contagem

```

Quando o arquivo tem pelo menos um erro léxico, o programa termina com código de saída 1. Sem erro, termina com 0. Com isso dá para usar o analisador em script ou num pipeline de integração contínua sem precisar ler a saída.

8. Vantagens e desvantagens

8.1 O que é bom

- **Aprende rápido.** Não tem linguagem de especificação separada nem passo de build. A gramática é Python e roda como Python, então o ciclo de editar, rodar e corrigir é imediato. Para quem está aprendendo, conta muito.
- **Avisa bem.** O PLY nasceu em sala de aula e confere a especificação com rigor. Ele aponta token declarado e nunca usado, não terminal inalcançável, regra que nunca reduz, produção cíclica e todos os conflitos, com o estado exato do autômato no `parser.out`. Muita coisa que em outras ferramentas só aparece como comportamento estranho durante a execução aqui aparece na hora de montar o parser.
- **Não depende de nada.** Dois arquivos Python, só biblioteca padrão.
- **O que se aprende serve em outro lugar.** Quem entende o PLY entende o modelo LALR(1) clássico e leva o aprendizado direto para Flex e Bison, JFlex e CUP ou Goyacc. O caminho contrário também funciona: uma gramática pronta em Yacc passa para o PLY quase sem esforço.
- **Depurar é tranquilo.** Ação semântica é uma função Python comum. Dá para usar o `pdb`, colocar `print` no meio, olhar a pilha e chamar qualquer coisa da biblioteca padrão dentro de uma redução. Num `.y` que gera C, é bem mais sofrido.

8.2 O que pesa contra

- **Projeto arquivado.** Repositório em somente leitura desde dezembro de 2025, sem manutenção, e o próprio autor recomendando outra ferramenta para projeto novo. Na disciplina, não faz diferença. Em produção, é uma decisão que precisa ser consciente, e o mínimo é copiar o código para dentro do projeto em vez de depender do PyPI.
- **Desempenho.** É Python puro. Comparado a um analisador gerado por Flex e Bison em C, fica uma ou duas ordens de grandeza atrás, e a própria documentação admite que ele serve para gramáticas de algumas centenas de regras, não para processar volume grande de entrada. Para ler vários gigabytes de log por dia, a escolha seria outra.
- **Gramática em docstring.** É elegante, mas prende a gramática a um detalhe do interpretador. Se o programa rodar com `python -OO`, as docstrings são removidas e o parser simplesmente não é construído. Linter e formatador também não enxergam nada ali, então um erro de digitação na gramática só aparece quando o código roda.
- **Limites do LALR(1).** Um token de lookahead não resolve tudo. O caso mais citado é o próprio C: saber se `"(x)*y"` é cast ou multiplicação depende de `x` ser ou não um typedef, e essa informação está na tabela de símbolos, não na gramática. A saída tradicional é o léxico consultar a tabela de símbolos, o chamado lexer hack. Dá para fazer no PLY. Fica feio, e quem esperava fases independentes leva um susto.
- **Tudo no nível do módulo.** A introspecção procura nomes no módulo inteiro, o que complica ter dois analisadores diferentes no mesmo programa e deixa estranho encapsular em classe. Foi o problema que o autor tentou resolver no SLY, o projeto seguinte dele.
- **Segurança.** O parâmetro `picklefile` do `yacc.yacc()` carrega a tabela com pickle, que executa código arbitrário na desserialização. O caso virou o CVE-2025-56005. É comportamento documentado, não bug acidental, e com o projeto arquivado não vai mudar. Na prática: nunca usar `picklefile` com arquivo de origem desconhecida.

9. O PLY e as alternativas de hoje

Biblioteca	Algoritmo	Quando é a melhor escolha
PLY	LALR(1)	Ensino de compiladores, portar gramática Yacc já existente e projeto pequeno que não pode ter dependência. Arquivado.
SLY	LALR(1)	Sucessor do PLY, mesmo autor, com API baseada em classe no lugar de variável global. Também não recebe mais manutenção ativa.
Lark	Earley e LALR(1)	Projeto ativo e a alternativa mais direta hoje. Gramática em arquivo próprio no formato EBNF, e o modo Earley aceita gramática ambígua que o LALR recusa.
pyparsing	PEG e descida recursiva	Formato pequeno, DSL e arquivo de configuração. Não tem etapa de construção de tabela, e é menos adequado para linguagem de programação completa.
ANTLR com alvo Python	ALL(*)	Projeto grande e multiplataforma, onde a mesma gramática gera analisador para Java, C#, Go e Python. Ferramenta mais pesada e exige JVM para gerar o código.
Descida recursiva escrita à mão	LL(k)	Quando se quer controle total sobre mensagem de erro e recuperação. É o caminho que vários compiladores industriais modernos seguiram.

Tabela 7. O PLY comparado às principais alternativas em Python.

Para o que a disciplina pede, que é aprender e demonstrar análise léxica e sintática, o PLY continua sendo uma boa escolha. A distância entre o que aparece no quadro e o que se escreve no código é curta. Para um produto que vai ficar anos no ar, o grupo iria de Lark ou ANTLR.

10. Considerações finais

Fazer o PLY funcionar leva minutos. O que tomou tempo foram as decisões de especificação, e quase todas eram invisíveis até darem errado. Uma regra fora de ordem, um quantificador guloso, um contador de linha que ninguém atualizou. Nenhum desses erros gera aviso. Eles só aparecem para quem olha a saída com cuidado.

O Elgol deixou isso ainda mais claro. A regra de identificador cabe numa linha de regex, e uma regex certa para as entradas válidas pode ser errada para as inválidas. Teste39 virando dois tokens corretos é o tipo de defeito que passa em qualquer teste feito só com exemplos bons. Foi a bateria com os casos ruins que mostrou o problema.

Isso não é exclusividade de compilador. Até o script de log escrito às pressas segue a mesma lógica: o que separa um parser bom de um ruim é o que ele faz com a entrada errada. Reconhecer a entrada certa é a parte fácil.

11. Referências

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

BEAZLEY, D. M. PLY (Python Lex-Yacc): documentação oficial. Disponível em: <https://www.dabeaz.com/ply/ply.html>.

BEAZLEY, D. M. dabeaz/ply: repositório do projeto, arquivado em 21 dez. 2025. Disponível em: <https://github.com/dabeaz/ply>.

BEAZLEY, D. M. PLY 4.0 documentation. Disponível em: <https://ply.readthedocs.io/>.

LEVINE, J.; MASON, T.; BROWN, D. lex & yacc. 2. ed. Sebastopol: O'Reilly, 1992.

PAXSON, V. et al. Flex: the fast lexical analyser generator. Disponível em: <https://github.com/westes/flex>.

PARR, T. The Definitive ANTLR 4 Reference. 2. ed. Raleigh: Pragmatic Bookshelf, 2013.

SHINAN, E. Lark: a parsing toolkit for Python. Disponível em: <https://github.com/lark-parser/lark>.

Apêndice A. Arquivos do projeto

O código completo acompanha o artigo. Tudo foi executado e conferido com Python 3.12 e PLY 3.11.

Arquivo	Conteúdo
lexico_elgol.py	Entrega da Etapa 1. Analisador léxico da linguagem Elgol, com validação de identificador, nome de função e número, lista de palavras reservadas, comentário, contagem de linha e coluna e diagnóstico detalhado de erro.
exemplo_professor.elg	Exemplo de código Elgol fornecido no enunciado, reproduzido sem alteração.
casos_borda.elg	Bateria com todos os casos válidos e inválidos citados no enunciado, usada para validar o analisador.
lexico_c.py	Analisador léxico para C do TR4, com as 32 palavras reservadas, constantes, operadores, comentários e diretivas.
sintatico_c.py	Analisador sintático LALR(1) para o subconjunto de C, com tabela de precedência, produção de erro e geração da AST.
teste.c e erro.c	Programas C usados no TR4, um válido e um propositalmente inválido.

Tabela 8. Arquivos que acompanham o artigo.

Para rodar o analisador da Etapa 1: `pip install ply` e depois `python lexico_elgol.py exemplo_professor.elg`.